# LAPORAN PRAKTIKUM PEMROGRAMAN WEB

# PERTEMUAN 7

# CRUD UPDATE, DELETE, PENCARIAN, DAN FILTER DENGAN LARAVEL


Disusun Oleh:

**ANDRIAN YUZA SWANDA**

**2401020157**

Dosen Pengampu:

**Nolan Efranda, M.Kom.**

**PROGRAM STUDI TEKNIK INFORMATIKA**

**FAKULTAS TEKNIK DAN TEKNOLOGI KEMARITIMAN**

**UNIVERSITAS MARITIM RAJA ALI HAJI**


**2026**

## I. TUJUAN

Pada praktikum ketujuh ini saya melengkapi aplikasi Laravel dari pertemuan keenam dengan operasi Update dan Delete, lalu menambahkan pencarian serta filter program studi. Update dipakai untuk mengubah data berdasarkan id, sedangkan Delete dipakai untuk menghapus satu data. Saya juga menguji seluruh rangkaian Create, Read, Update, dan Delete, termasuk kasus email duplikat dan kegagalan query.

Sasaran praktikum ini adalah memakai prepared statement untuk UPDATE dan DELETE, memvalidasi masukan di sisi server, menjaga agar NIM atau email tidak bertabrakan dengan mahasiswa lain, memakai CSRF dan method spoofing, serta mencatat masalah dan perbaikan melalui GitHub Issue dan commit Git.

## II. IDENTIFIKASI MASALAH

Setelah Modul 6, halaman /mahasiswa baru bisa menambah dan menampilkan data. Belum ada tombol Ubah dan Hapus, belum ada pencarian, dan belum ada filter program studi. Saya juga menemukan masalah integrasi: ketika pencarian atau filter sedang aktif, redirect setelah Update dan Delete kembali ke /mahasiswa tanpa query string, sehingga filter yang sedang dipakai hilang. Masalah tersebut saya catat sebagai issue GitHub nomor 1.

Perangkat yang saya gunakan masih sama seperti pertemuan sebelumnya: Laravel dengan PHP 8.2.12 dan ekstensi pdo_mysql, MySQL/MariaDB pada XAMPP, Visual Studio Code, PowerShell, Google Chrome, server lokal php artisan serve pada http://127.0.0.1:8001, serta Git dan GitHub untuk menyimpan commit dan issue.

## III. DATA AWAL

Praktik ini saya mulai dari empat data mahasiswa yang sudah ada sejak Modul 6, termasuk Rina Kurnia. Saya memeriksa data dan kolom id dengan query berikut sebelum mengubah route.

```
USE praktikum_web_2401020157;
SELECT m.id, m.nim, m.nama, m.email, m.usia, p.nama_prodi
FROM mahasiswa AS m
JOIN program_studi AS p ON p.id = m.program_studi_id
ORDER BY m.nim;
```

| ID | NIM | Nama | Email | Usia | Program Studi |
|---|---|---|---|---|---|
| 5 | 2201010003 | Rina Kurnia | rina@example.com | 20 | Teknik Informatika |
| 3 | 2401020045 | Muhammad Fauzi | fauzi@example.com | 19 | Sistem Informasi |
| 2 | 2401020151 | Christian Aprilio Sihite | christian@example.com | 20 | Teknik Informatika |
| 1 | 2401020157 | Andrian Yuza Swanda | andrian.yuza@example.com | 21 | Teknik Informatika |

Data Modul 6 saya pertahankan agar laporan tetap nyambung dengan praktikum sebelumnya. Pada pengujian Create dan Delete saya menambahkan satu data sementara, Budi Pratama, lalu menghapusnya kembali sehingga empat data awal tetap utuh.

## IV. RANCANGAN ROUTE

Saya memakai lima route utama. Route GET /mahasiswa diperluas agar menerima parameter q dan program_studi_id. Route edit menampilkan data lama, route PUT menyimpan perubahan, dan route DELETE menghapus data. Form edit dan hapus tetap mengirim POST, lalu method spoofing mengubahnya menjadi PUT atau DELETE.

| Method | URI | Nama route | Fungsi |
|---|---|---|---|
| GET | /mahasiswa | mahasiswa.index | Read, pencarian, dan filter |
| POST | /mahasiswa | mahasiswa.store | Create dari Modul 6 |
| GET | /mahasiswa/{id}/edit | mahasiswa.edit | Membuka form edit |
| PUT | /mahasiswa/{id} | mahasiswa.update | Validasi dan Update |
| DELETE | /mahasiswa/{id} | mahasiswa.destroy | Delete berdasarkan id |

Form edit dan form hapus tetap dikirim dengan method POST karena HTML belum mengenal PUT dan DELETE. Di dalam form saya menambahkan @csrf untuk token keamanan dan @method('PUT') atau @method('DELETE') agar Laravel membaca request sesuai method yang dimaksud. Tanpa token CSRF, Laravel menolak request dengan status 419.

| Kolom | Aturan validasi | Pesan bila gagal |
|---|---|---|
| nim | required, string, max:20 | NIM wajib diisi. |
| nama | required, string, max:100 | Nama wajib diisi. |
| email | required, email, max:100 | Email wajib diisi / format tidak valid. |
| usia | required, integer, 15-100 | Usia wajib diisi / harus angka. |
| program_studi_id | required, integer | Program studi wajib dipilih. |

Aturan pada tabel di atas dipakai oleh route POST (Create) dan route PUT (Update). Khusus Update, pengecekan duplikat menambahkan kondisi id <> :id agar data yang sedang diubah tidak dianggap bentrok dengan dirinya sendiri.

Rencana pengujian saya susun mengikuti urutan CRUD: menambah satu data uji, membaca seluruh data, mencari dengan kata kunci dan filter, mengubah data, mencoba email duplikat, lalu menghapus data uji. Setiap langkah saya catat hasilnya pada bagian IX agar bukti pengujian mudah diperiksa.

Alur integrasi CRUD yang saya pakai digambarkan seperti berikut. Filter tetap dibawa pada tautan Ubah dan form Hapus supaya setelah redirect pengguna tidak kehilangan konteks pencarian.

```
Browser buka /mahasiswa?q=...&program_studi_id=...
  -> Route GET menyusun WHERE 1 = 1, menambah kondisi pencarian/filter
  -> prepared statement dengan placeholder unik
  -> Blade menampilkan form Create, form pencarian, tabel, aksi Ubah/Hapus
  -> Ubah: GET /mahasiswa/{id}/edit -> form POST + @method('PUT')
  -> validasi, cek duplikat dengan id <> :id, UPDATE, redirect + filter
  -> Hapus: POST + @method('DELETE')
  -> DELETE berdasarkan id, rowCount(), redirect + feedback
```

## V. KODE UPDATE

Route edit membaca satu data berdasarkan id beserta daftar program studi. Query filter q dan program_studi_id juga dibaca agar nilai filter bisa dibawa ke form dan dikembalikan setelah penyimpanan. Route edit lengkap saya tampilkan berikut.

```
Route::get('/mahasiswa/{id}/edit', function (
    Request $request,
    int $id
) {
    // Filter dibawa agar tetap aktif setelah data diubah
    $kataKunci = trim((string) $request->query('q', ''));
    $programStudiId = $request->query('program_studi_id', '');
    try {
        $pdo = DB::connection()->getPdo();
        $ambilMahasiswa = $pdo->prepare(
            'SELECT id, nim, nama, email, usia, program_studi_id
             FROM mahasiswa WHERE id = :id'
        );
        $ambilMahasiswa->execute(['id' => $id]);
        $mahasiswa = $ambilMahasiswa->fetch(\PDO::FETCH_ASSOC);
        if (!$mahasiswa) {
            return redirect()->route('mahasiswa.index')->with(
                'gagal',
                'Data mahasiswa tidak ditemukan.'
            );
        }
        $ambilProdi = $pdo->prepare(
            'SELECT id, nama_prodi
             FROM program_studi
             ORDER BY nama_prodi'
        );
        $ambilProdi->execute();
        $daftarProgramStudi = $ambilProdi->fetchAll(
            \PDO::FETCH_ASSOC
        );
        return view('mahasiswa-edit', compact(
            'mahasiswa',
            'daftarProgramStudi',
            'kataKunci',
            'programStudiId'
        ));
    } catch (\Throwable $error) {
        report($error);
        return redirect()->route('mahasiswa.index')->with(
            'gagal',
            'Data gagal dibuka untuk diubah.'
        );
    }
})->name('mahasiswa.edit');
```

Route PUT memakai validasi yang sama dengan Create. Pemeriksaan duplikat memakai kondisi id <> :id agar data yang sedang diubah tidak dianggap duplikat dirinya sendiri. Setelah UPDATE berhasil, pengguna diarahkan kembali ke daftar dengan filter yang sama.

```
Route::put('/mahasiswa/{id}', function (
    Request $request,
    int $id
) {
    $data = $request->validate([
        'nim' => ['required', 'string', 'max:20'],
        'nama' => ['required', 'string', 'max:100'],
        'email' => ['required', 'email', 'max:100'],
        'usia' => ['required', 'integer', 'min:15', 'max:100'],
        'program_studi_id' => ['required', 'integer'],
    ], [
        'nim.required' => 'NIM wajib diisi.',
        'nama.required' => 'Nama wajib diisi.',
        'email.required' => 'Email wajib diisi.',
        'email.email' => 'Format email tidak valid.',
        'usia.required' => 'Usia wajib diisi.',
        'usia.integer' => 'Usia harus berupa angka.',
        'program_studi_id.required' => 'Program studi wajib dipilih.',
    ]);
    // Simpan filter yang sedang aktif untuk redirect setelah Update
    $filter = array_filter([
        'q' => trim((string) $request->input('q', '')),
        'program_studi_id' => $request->input('program_studi_id', ''),
    ], fn ($nilai) => $nilai !== '' && $nilai !== null);
```

Lanjutan route PUT:

```
    try {
        $pdo = DB::connection()->getPdo();
        // Cek duplikat milik mahasiswa lain, bukan data yang diubah
        $cekDuplikat = $pdo->prepare(
            'SELECT COUNT(*) FROM mahasiswa
             WHERE (nim = :nim OR email = :email)
               AND id <> :id'
        );
        $cekDuplikat->execute([
            'nim' => $data['nim'],
            'email' => $data['email'],
            'id' => $id,
        ]);
        if ((int) $cekDuplikat->fetchColumn() > 0) {
            return back()->withInput()->with(
                'gagal',
                'NIM atau email sudah digunakan mahasiswa lain.'
            );
        }
```

Lanjutan route PUT, bagian UPDATE dan penanganan error:

```
        $ubah = $pdo->prepare(
            'UPDATE mahasiswa
             SET nim = :nim, nama = :nama, email = :email,
                 usia = :usia, program_studi_id = :program_studi_id
             WHERE id = :id'
        );
        $ubah->execute([
            'nim' => $data['nim'],
            'nama' => $data['nama'],
            'email' => $data['email'],
            'usia' => $data['usia'],
            'program_studi_id' => $data['program_studi_id'],
            'id' => $id,
        ]);
        return redirect()->route('mahasiswa.index', $filter)->with(
            'berhasil',
            'Data mahasiswa berhasil diperbarui.'
        );
    } catch (\Throwable $error) {
        report($error);
        return back()->withInput()->with(
            'gagal',
            'Data gagal diperbarui. Periksa koneksi dan data.'
        );
    }
})->name('mahasiswa.update');
```

## VI. KODE DELETE

Delete hanya dijalankan setelah form mengirim token CSRF dan method spoofing DELETE. Prepared statement menghapus data berdasarkan id, lalu rowCount() dipakai untuk membedakan data yang benar-benar terhapus dengan id yang tidak ditemukan.

```
Route::delete('/mahasiswa/{id}', function (
    Request $request,
    int $id
) {
    // Simpan filter yang sedang aktif untuk redirect setelah Delete
    $filter = array_filter([
        'q' => trim((string) $request->input('q', '')),
        'program_studi_id' => $request->input('program_studi_id', ''),
    ], fn ($nilai) => $nilai !== '' && $nilai !== null);
    try {
        $pdo = DB::connection()->getPdo();
        $hapus = $pdo->prepare(
            'DELETE FROM mahasiswa WHERE id = :id'
        );
        $hapus->execute(['id' => $id]);
```

Lanjutan route DELETE, pengecekan hasil dan penanganan error:

```
        if ($hapus->rowCount() === 0) {
            return redirect()->route('mahasiswa.index', $filter)->with(
                'gagal',
                'Data mahasiswa tidak ditemukan.'
            );
        }
        return redirect()->route('mahasiswa.index', $filter)->with(
            'berhasil',
            'Data mahasiswa berhasil dihapus.'
        );
    } catch (\Throwable $error) {
        report($error);
        return redirect()->route('mahasiswa.index', $filter)->with(
            'gagal',
            'Data gagal dihapus. Periksa relasi atau koneksi.'
        );
    }
})->name('mahasiswa.destroy');
```

## VII. KODE PENCARIAN DAN FILTER

Route GET menyusun SQL secara bertahap dengan WHERE 1 = 1. Setiap placeholder pencarian diberi nama berbeda agar kompatibel dengan native prepared statement PDO. Filter program studi hanya ditambahkan ketika parameternya berisi nilai.

```
Route::get('/mahasiswa', function (Request $request) {
    $kataKunci = trim((string) $request->query('q', ''));
    $programStudiId = $request->query('program_studi_id', '');
    try {
        // Mengambil koneksi PDO dari konfigurasi Laravel (.env)
        $pdo = DB::connection()->getPdo();
        // Read: data mahasiswa beserta nama program studi
        $sql = 'SELECT m.id, m.nim, m.nama, m.email, m.usia,
                       m.program_studi_id, p.nama_prodi
                FROM mahasiswa AS m
                JOIN program_studi AS p
                  ON p.id = m.program_studi_id
                WHERE 1 = 1';
        $parameter = [];
        if ($kataKunci !== '') {
            // Placeholder berbeda agar kompatibel dengan native PDO
            $sql .= ' AND (m.nim LIKE :kata_kunci_nim
                      OR m.nama LIKE :kata_kunci_nama
                      OR m.email LIKE :kata_kunci_email)';
            $nilaiPencarian = '%' . $kataKunci . '%';
            $parameter['kata_kunci_nim'] = $nilaiPencarian;
            $parameter['kata_kunci_nama'] = $nilaiPencarian;
            $parameter['kata_kunci_email'] = $nilaiPencarian;
        }
```

Lanjutan route GET, filter program studi sampai penanganan error:

```
        if ($programStudiId !== '') {
            $sql .= ' AND m.program_studi_id = :program_studi_id';
            $parameter['program_studi_id'] = (int) $programStudiId;
        }
        $sql .= ' ORDER BY m.nim';
        $ambilMahasiswa = $pdo->prepare($sql);
        $ambilMahasiswa->execute($parameter);
        $daftarMahasiswa = $ambilMahasiswa->fetchAll(
            \PDO::FETCH_ASSOC
        );
        // Read: daftar program studi untuk pilihan pada form
        $ambilProdi = $pdo->prepare(
            'SELECT id, nama_prodi
             FROM program_studi
             ORDER BY nama_prodi'
        );
        $ambilProdi->execute();
        $daftarProgramStudi = $ambilProdi->fetchAll(
            \PDO::FETCH_ASSOC
        );
        return view('mahasiswa', compact(
            'daftarMahasiswa',
            'daftarProgramStudi',
            'kataKunci',
            'programStudiId'
        ));
    } catch (\Throwable $error) {
        report($error);
        return response(
            'Data tidak dapat dibaca. Periksa koneksi dan query.',
            500
        );
    }
})->name('mahasiswa.index');
```

Placeholder :kata_kunci_nim, :kata_kunci_nama, :kata_kunci_email, dan :program_studi_id diisi melalui array $parameter. Cara ini membuat nilai pencarian diperlakukan sebagai data, bukan bagian dari perintah SQL, sehingga pencarian tetap aman. Ketika pengguna mengosongkan form, kondisi WHERE tidak ditambahkan dan seluruh data ditampilkan kembali.

## VIII. KODE BLADE

Form pencarian memakai method GET dan mempertahankan nilai q serta program studi yang sedang dipilih. Tampilan tetap polos tanpa CSS.

```
<h2>Pencarian dan Filter</h2>
    <form action="{{ route('mahasiswa.index') }}" method="GET">
        <p>Kata Kunci<br>
            <input type="text" name="q" value="{{ $kataKunci }}">
        </p>
        <p>Program Studi<br>
            <select name="program_studi_id">
                <option value="">-- Semua Program Studi --</option>
                @foreach ($daftarProgramStudi as $programStudi)
                    <option value="{{ $programStudi['id'] }}"
                        @selected((string) $programStudiId === (string)
                            $programStudi['id'])>
                        {{ $programStudi['nama_prodi'] }}
                    </option>
                @endforeach
            </select>
        </p>
        <button type="submit">Cari</button>
        <a href="{{ route('mahasiswa.index') }}">Reset</a>
    </form>
```

Tabel menampilkan aksi Ubah dan Hapus. Bagian awal tabel berisi pembentuk parameter filter dan kepala tabel seperti berikut.

```
<h2>Daftar Mahasiswa</h2>
    @php
        $parameterFilter = array_filter([
            'q' => $kataKunci,
            'program_studi_id' => $programStudiId,
        ], fn ($nilai) => $nilai !== '' && $nilai !== null);
    @endphp
    <table border="1" cellpadding="6" cellspacing="0">
        <tr>
            <th>NIM</th><th>Nama</th><th>Email</th>
            <th>Usia</th><th>Program Studi</th><th>Aksi</th>
        </tr>
        @forelse ($daftarMahasiswa as $mahasiswa)
            <tr>
                <td>{{ $mahasiswa['nim'] }}</td>
                <td>{{ $mahasiswa['nama'] }}</td>
                <td>{{ $mahasiswa['email'] }}</td>
                <td>{{ $mahasiswa['usia'] }}</td>
                <td>{{ $mahasiswa['nama_prodi'] }}</td>
                <td>
```

Lanjutan tabel, aksi Ubah/Hapus beserta hidden input filter:

```
                    <a href="{{ route('mahasiswa.edit', array_merge(['id' =>
                        $mahasiswa['id']], $parameterFilter)) }}">
                        Ubah
                    </a>
                    <form action="{{ route('mahasiswa.destroy',
                        $mahasiswa['id']) }}"
                          method="POST">
                        @csrf
                        @method('DELETE')
                        @if ($kataKunci !== '')
                            <input type="hidden" name="q" value="{{ $kataKunci
                                }}">
                        @endif
                        @if ($programStudiId !== '')
                            <input type="hidden" name="program_studi_id"
                                   value="{{ $programStudiId }}">
                        @endif
                        <button type="submit"
                            onclick="return confirm('Hapus data ini?')">
                            Hapus
                        </button>
                    </form>
                </td>
            </tr>
        @empty
            <tr><td colspan="6">Data tidak ditemukan.</td></tr>
        @endforelse
    </table>
</body>
</html>
```

@forelse dipakai agar tabel tetap rapi ketika hasil query kosong; cabang @empty menampilkan baris "Data tidak ditemukan." Tanpa @forelse, tabel akan kehilangan struktur saat pencarian tidak menemukan data. Tombol Hapus memakai onclick="return confirm(...)" supaya pengguna mendapat konfirmasi sebelum data dihapus.

Tautan Ubah memakai array_merge agar id mahasiswa dan parameter filter dikirim bersama. Dengan cara ini halaman edit tahu filter mana yang sedang aktif, lalu mengembalikannya setelah penyimpanan. Form Hapus membawa nilai yang sama melalui hidden input q dan program_studi_id.

Halaman edit memakai nilai lama dari database sebagai nilai awal. Helper old() dipakai agar isian yang baru diketik tetap tampil ketika validasi gagal, dan hidden input menjaga filter tetap aktif.

```
<form action="{{ route('mahasiswa.update', $mahasiswa['id']) }}"
          method="POST">
        @csrf
        @method('PUT')
        @if ($kataKunci !== '')
            <input type="hidden" name="q"
                   value="{{ old('q', $kataKunci) }}">
        @endif
        @if ($programStudiId !== '')
            <input type="hidden" name="program_studi_id"
                   value="{{ old('program_studi_id', $programStudiId) }}">
        @endif
        <p>NIM<br>
            <input type="text" name="nim"
                   value="{{ old('nim', $mahasiswa['nim']) }}">
        </p>
        <p>Nama<br>
            <input type="text" name="nama"
                   value="{{ old('nama', $mahasiswa['nama']) }}">
        </p>
        <p>Email<br>
            <input type="email" name="email"
                   value="{{ old('email', $mahasiswa['email']) }}">
        </p>
```

Lanjutan form edit, pilihan program studi dan tombol simpan:

```
        <p>Usia<br>
            <input type="number" name="usia"
                   value="{{ old('usia', $mahasiswa['usia']) }}">
        </p>
        <p>Program Studi<br>
            <select name="program_studi_id">
                @foreach ($daftarProgramStudi as $programStudi)
                    <option value="{{ $programStudi['id'] }}"
                        @selected(old(
                            'program_studi_id',
                            $mahasiswa['program_studi_id']
                        ) == $programStudi['id'])>
                        {{ $programStudi['nama_prodi'] }}
                    </option>
                @endforeach
            </select>
        </p>
        <button type="submit">Simpan Perubahan</button>
        <a href="{{ route('mahasiswa.index') }}">Kembali</a>
    </form>
```

## IX. SKENARIO DAN HASIL UJI CRUD

Saya menguji tujuh skenario sesuai modul, ditambah satu pengujian khusus untuk memastikan filter tetap aktif setelah Update. Data uji Create dan Delete memakai Budi Pratama, sedangkan data uji Update memakai Rina.

| Keperluan | NIM | Nama | Email | Usia | Program Studi |
|---|---|---|---|---|---|
| Create/Delete | 2201020001 | Budi Pratama | budi@example.com | 21 | Sistem Informasi |
| Update | 2201010003 | Rina Kurniawati | rina.kurniawati@example.com | 21 | Teknik Informatika |
| Duplikat | - | - | fauzi@example.com | - | - |

Hasil tiap skenario saya catat pada tabel berikut. Jumlah data dihitung tanpa baris header tabel.

| No | Skenario | Hasil |
|---|---|---|
| 1 | Create data valid Budi Pratama | Pesan "Data mahasiswa berhasil ditambahkan." tampil dan tabel menjadi lima data. |
| 2 | Read tanpa filter | Empat data awal tampil berurutan berdasarkan NIM dan tombol Ubah/Hapus tersedia. |
| 3 | Cari Rina + filter Teknik Informatika | Hanya Rina tampil; URL memuat q=Rina dan program_studi_id=1. |
| 4 | Update Rina | Pesan "Data mahasiswa berhasil diperbarui." tampil; nama, email, dan usia berubah; filter tetap aktif. |
| 5 | Update dengan email duplikat | Update ditolak dengan pesan "NIM atau email sudah digunakan mahasiswa lain."; nilai form tetap tampil. |
| 6 | Delete Budi | Pesan "Data mahasiswa berhasil dihapus." tampil dan jumlah data kembali menjadi empat. |
| 7 | Koneksi atau query gagal | Status 500 dengan pesan aman "Data tidak dapat dibaca. Periksa koneksi dan query."; detail dicatat report(). |
| 8 | Update saat filter aktif (issue #1) | URL tetap memuat q=Rina&program_studi_id=1 dan tabel tetap menampilkan data yang difilter. |

## X. TANGKAPAN LAYAR

Tangkapan layar di bawah ini saya ambil dari aplikasi yang berjalan di http://127.0.0.1:8001. Setiap gambar mengikuti urutan skenario pada tabel sebelumnya.

# Data Mahasiswa

## Tambah Mahasiswa

NIM

Nama

Email

Usia

Program Studi
-- Pilih Program Studi --

Tambah Data

## Pencarian dan Filter

Kata Kunci

Program Studi
-- Semua Program Studi --

Cari Reset

## Daftar Mahasiswa

| NIM | Nama | Email | Usia | Program Studi | Aksi |
|---|---|---|---|---|---|
| 2201010003 | Rina Kurnia | rina@example.com | 20 | Teknik Informatika | Ubah Hapus |
| 2401020045 | Muhammad Fauzi | fauzi@example.com | 19 | Sistem Informasi | Ubah Hapus |
| 2401020151 | Christian Aprilio Sihite | christian@example.com | 20 | Teknik Informatika | Ubah Hapus |
| 2401020157 | Andrian Yuza Swanda | andrian.yuza@example.com | 21 | Teknik Informatika | Ubah Hapus |

*Gambar 1. Halaman indeks dengan empat data Modul 6, form pencarian, dan aksi Ubah/Hapus.*

# Data Mahasiswa

Data mahasiswa berhasil ditambahkan.

## Tambah Mahasiswa

NIM

Nama

Email

Usia

Program Studi
-- Pilih Program Studi --

Tambah Data

## Pencarian dan Filter

Kata Kunci

Program Studi
-- Semua Program Studi --

Cari Reset

## Daftar Mahasiswa

| NIM | Nama | Email | Usia | Program Studi | Aksi |
|---|---|---|---|---|---|
| 2201010003 | Rina Kurnia | rina@example.com | 20 | Teknik Informatika | Ubah Hapus |
| 2201020001 | Budi Pratama | budi@example.com | 21 | Sistem Informasi | Ubah Hapus |
| 2401020045 | Muhammad Fauzi | fauzi@example.com | 19 | Sistem Informasi | Ubah Hapus |
| 2401020151 | Christian Aprilio Sihite | christian@example.com | 20 | Teknik Informatika | Ubah Hapus |
| 2401020157 | Andrian Yuza Swanda | andrian.yuza@example.com | 21 | Teknik Informatika | Ubah Hapus |

*Gambar 2. Create data valid Budi Pratama berhasil; tabel menjadi lima data.*

# Data Mahasiswa

## Tambah Mahasiswa

NIM

Nama

Email

Usia

Program Studi
-- Pilih Program Studi --

Tambah Data

## Pencarian dan Filter

Kata Kunci
Rina

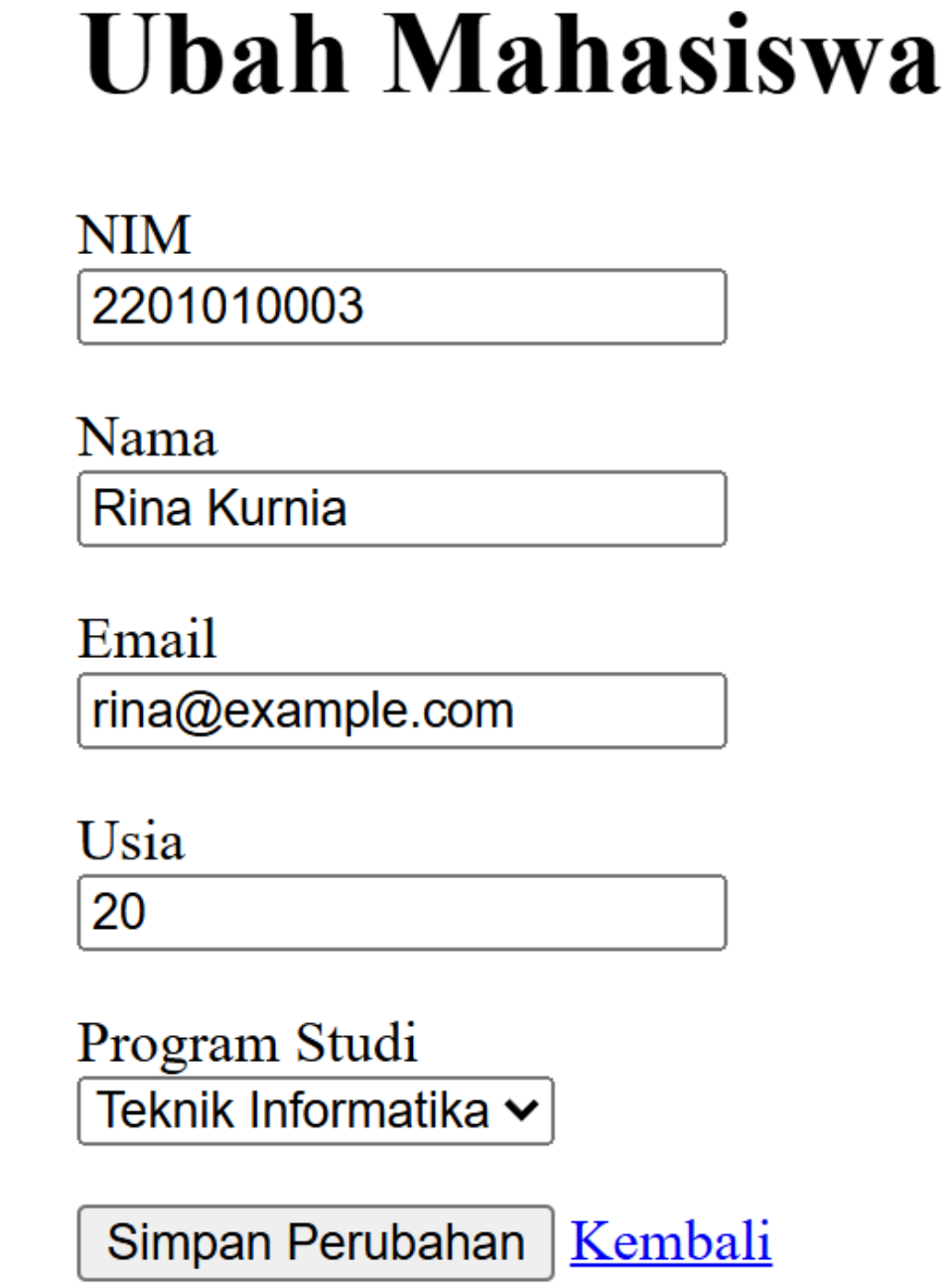
Program Studi
Teknik Informatika

Cari Reset

## Daftar Mahasiswa

| NIM | Nama | Email | Usia | Program Studi | Aksi |
|---|---|---|---|---|---|
| 2201010003 | Rina Kurnia | rina@example.com | 20 | Teknik Informatika | Ubah Hapus |

*Gambar 3. Pencarian Rina dengan filter Teknik Informatika menampilkan satu data.*




*Gambar 4. Form edit Rina terisi nilai lama dan memakai method spoofing PUT.*

# Data Mahasiswa

Data mahasiswa berhasil diperbarui.

## Tambah Mahasiswa

NIM

Nama

Email

Usia

Program Studi
-- Pilih Program Studi --

Tambah Data

## Pencarian dan Filter

Kata Kunci
Rina

Program Studi
Teknik Informatika

Cari Reset

## Daftar Mahasiswa

| NIM | Nama | Email | Usia | Program Studi | Aksi |
|---|---|---|---|---|---|
| 2201010003 | Rina Kurniawati | rina.kurniawati@example.com | 21 | Teknik Informatika | Ubah Hapus |

*Gambar 5. Update berhasil; data Rina berubah dan filter q=Rina&program_studi_id=1 tetap aktif.*




*Gambar 6. Update dengan email milik Muhammad Fauzi ditolak dan isian form tetap tampil.*

# Data Mahasiswa

Data mahasiswa berhasil dihapus.

## Tambah Mahasiswa

NIM

Nama

Email

Usia

Program Studi
-- Pilih Program Studi --

Tambah Data

## Pencarian dan Filter

Kata Kunci

Program Studi
-- Semua Program Studi --

Cari Reset

## Daftar Mahasiswa

| NIM | Nama | Email | Usia | Program Studi | Aksi |
|---|---|---|---|---|---|
| 2201010003 | Rina Kurniawati | rina.kurniawati@example.com | 21 | Teknik Informatika | Ubah Hapus |
| 2401020045 | Muhammad Fauzi | fauzi@example.com | 19 | Sistem Informasi | Ubah Hapus |
| 2401020151 | Christian Aprilio Sihite | christian@example.com | 20 | Teknik Informatika | Ubah Hapus |
| 2401020157 | Andrian Yuza Swanda | andrian.yuza@example.com | 21 | Teknik Informatika | Ubah Hapus |

*Gambar 7. Delete Budi berhasil; pesan tampil dan jumlah data kembali menjadi empat.*

Data tidak dapat dibaca. Periksa koneksi dan query.

*Gambar 8. Kegagalan koneksi menampilkan pesan aman dari blok catch.*

## XI. ERROR DAN DEBUGGING

Masalah pertama muncul ketika saya menguji Update dari halaman hasil pencarian. Setelah tombol Simpan Perubahan ditekan, data berhasil berubah, tetapi URL kembali ke /mahasiswa tanpa q dan program_studi_id. Akibatnya tabel menampilkan seluruh data dan pengguna kehilangan konteks pencarian. Saya telusuri penyebabnya: form edit hanya mengirim data mahasiswa, sedangkan route Update melakukan redirect ke route index tanpa parameter. Perbaikannya adalah menambahkan hidden input q dan program_studi_id pada form edit dan form hapus, membaca query filter di route edit, lalu mengirimkan filter tersebut pada redirect Update dan Delete. Perbaikan ini saya catat pada issue #1 dan commit ad99e3b.

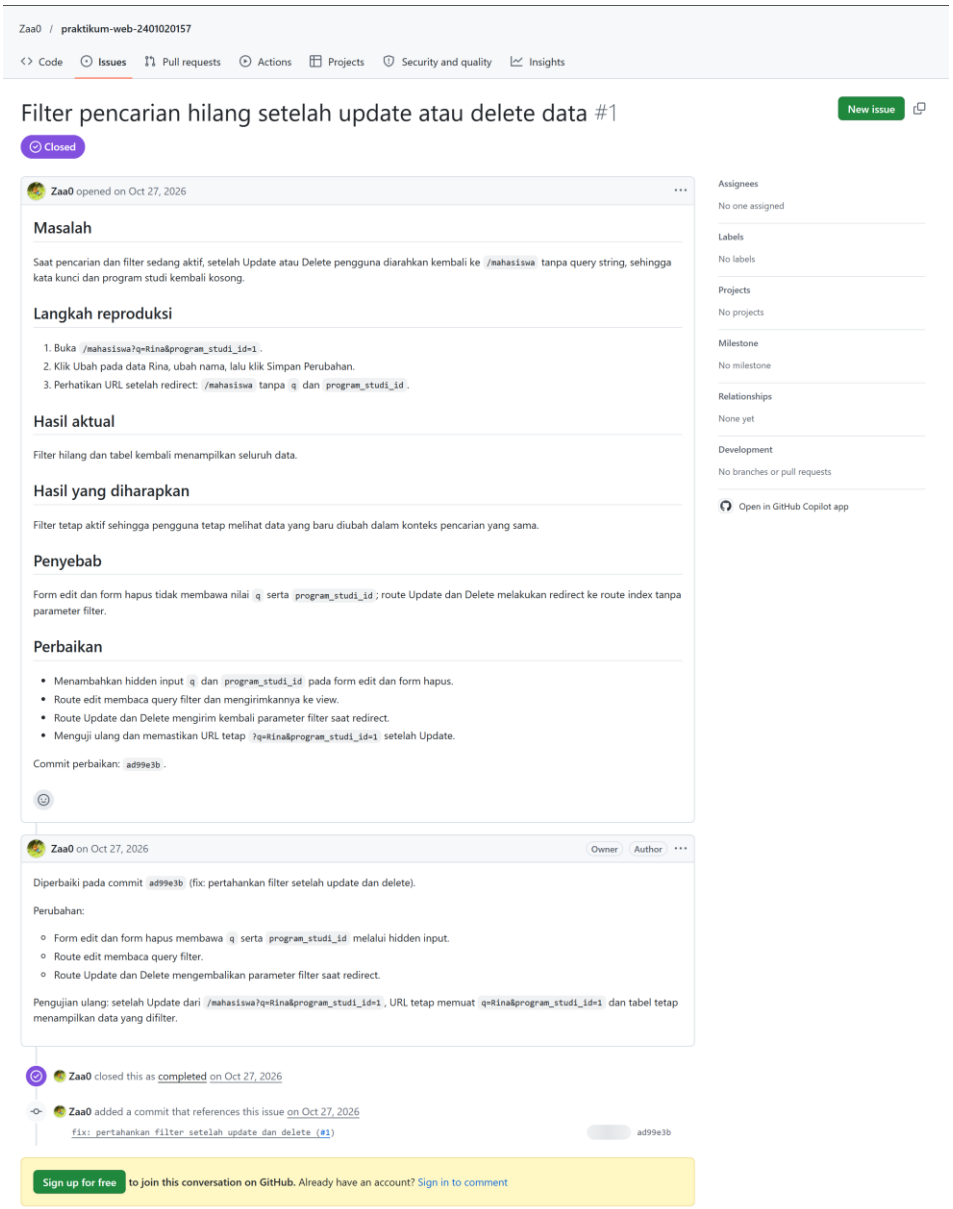
Masalah kedua adalah pengujian kegagalan query. Ketika nama basis data saya ubah menjadi salah, halaman debug Laravel muncul lebih dahulu karena session disimpan di basis data. Untuk menguji blok catch, saya mengganti SESSION_DRIVER menjadi file, menjalankan php artisan config:clear, lalu membuka /mahasiswa. Pesan aman dan status 500 muncul sesuai harapan. Setelah pengujian, konfigurasi saya kembalikan ke database.

Pada Update, saya juga memastikan pemeriksaan duplikat memakai id <> :id. Tanpa kondisi itu, data yang sedang diubah akan dianggap duplikat dirinya sendiri. Saya mengujinya dengan mengirim email milik Muhammad Fauzi; sistem menolak perubahan dan mempertahankan isian form melalui back()->withInput() serta helper old().

## XII. GITHUB ISSUE

Saya membuat issue GitHub nomor 1 berjudul "Filter pencarian hilang setelah update atau delete data". Issue berisi masalah, langkah reproduksi, hasil aktual, hasil yang diharapkan, penyebab, dan rencana perbaikan. Setelah perbaikan diuji, issue saya tutup dengan komentar yang menyebut commit ad99e3b.

1. URL issue: https://github.com/Zaa0/praktikum-web-2401020157/issues/1
2. Status: Closed (completed).
3. Commit penyelesai: ad99e3b - fix: pertahankan filter setelah update dan delete.




*Gambar 9. Issue #1 berisi masalah filter dan sudah ditutup setelah perbaikan.*

## XIII. COMMIT LOG

Perubahan saya simpan dalam dua commit kode yang terpisah, lalu laporan ini disimpan pada commit dokumentasi. Commit pertama memuat integrasi Update, Delete, pencarian, dan filter. Commit kedua memuat perbaikan filter yang ditemukan saat pengujian dan dirujuk oleh issue #1.

| Hash | Pesan commit | Isi |
|---|---|---|
| ed6ca78 | feat: tambah update delete dan pencarian filter mahasiswa | Route edit, PUT, DELETE, pencarian, filter, dan tampilan Blade. |
| ad99e3b | fix: pertahankan filter setelah update dan delete (#1) | Hidden input filter, query filter di route edit, dan redirect Update/Delete. |

```
PS D:\praktikum_web_2401020157> git log --oneline -5
ad99e3b fix: pertahankan filter setelah update dan delete (#1)
ed6ca78 feat: tambah update delete dan pencarian filter mahasiswa
a2ffc09 Pertemuan 6 CRUD Create Read
b024422 Laporan pertemuan 5
4a9c9df Tambah komentar route PDO pertemuan 5
```

*Gambar 10. Commit log dua commit Modul 7 pada repositori.*

## XIV. KENDALA DAN SOLUSI

Kendala pertama adalah filter yang hilang setelah Update dan Delete. Solusinya adalah membawa q dan program_studi_id melalui hidden input, membacanya di route edit, dan mengirimkannya kembali pada redirect. Kendala kedua adalah halaman debug yang menutupi pesan error saat koneksi dibuat gagal; solusinya memakai session driver file khusus saat pengujian, lalu mengembalikannya ke database.

Kendala ketiga adalah perbedaan data contoh pada modul. Modul memakai Andi, Siti, Budi, dan Rina, sedangkan basis data saya berisi empat data hasil Modul 6. Saya mempertahankan data asli dan menambahkan Budi Pratama hanya sebagai data uji Create/Delete agar laporan sesuai dengan keadaan aplikasi yang sebenarnya.

## XV. KESIMPULAN

Praktikum ketujuh ini berhasil melengkapi operasi CRUD pada aplikasi Laravel saya. Update dan Delete berjalan dengan validasi, prepared statement, CSRF, method spoofing, pengecekan duplikat yang mengecualikan id sendiri, serta feedback yang jelas. Pencarian NIM, nama, dan email dapat digabung dengan filter program studi, dan filter tetap aktif setelah Update maupun Delete. Seluruh skenario utama, termasuk email duplikat dan kegagalan query, sudah diuji; masalah integrasi tercatat pada issue #1 dan diselesaikan melalui commit ad99e3b.

## XVI. TAUTAN REPOSITORY DAN LAMPIRAN SOURCE CODE

1. Repository GitHub: https://github.com/Zaa0/praktikum-web-2401020157
2. Issue GitHub: https://github.com/Zaa0/praktikum-web-2401020157/issues/1
3. Commit Modul 7: ed6ca78 dan ad99e3b.
4. Source code yang dikumpulkan:
    - routes/web.php: route Read dengan pencarian/filter, Create, edit, Update, dan Delete berbasis PDO prepared statement.
    - resources/views/mahasiswa.blade.php: form Create, form pencarian/filter, tabel, dan aksi Ubah/Hapus.
    - resources/views/mahasiswa-edit.blade.php: form edit dengan CSRF, method spoofing PUT, old(), dan filter aktif.
    - .env.example: dokumentasi variabel koneksi tanpa kredensial rahasia (file .env tidak di-commit).